ICT103 Programming

Topic 8
Methods

CIHE Australia

Resources

Prescribed Texts

- McMullen, K., Matthews, E., & Parsons, J. J. (2022). Programming with Python (1st ed.)., Cengage Learning

Recommended Readings

- Lutz, M. (2013). Learning Python (5th ed.). O'Reilly Media, Inc

Lesson Learned from Week 7:

- Understand the difference between object-oriented programming (OOP) and procedural programming
- Exploring concepts of objects and classes in OOP
- Understanding the use of static elements in a Class
- Characteristics of Objects in OOP

Week 8: Lesson Learning Outcomes

- Understand methods and their purpose
- Explain built-in overriding
- Use of Constructors in OOP
- Identify the visibility of variables in OOP

Why use Methods?

- **Method**:
  - A block of code that performs some action related to its object
  - Like a function that belongs to an object and performs a specific task, routine, operation, or process
  - Methods help organise repetitive tasks related to an object
  - Without methods, objects would just be a collection of related data

- If a method returns a value, you set the value to be equal to the method call
- Otherwise, you simply call the method
- To become methods, functions must be declared inside a class definition (indented) and take at least one parameter, the keyword self

Method Anatomy

Class header

Method definition

```
class Customer:

    def identify(self, name):
        print("I am Customer " + name)
```

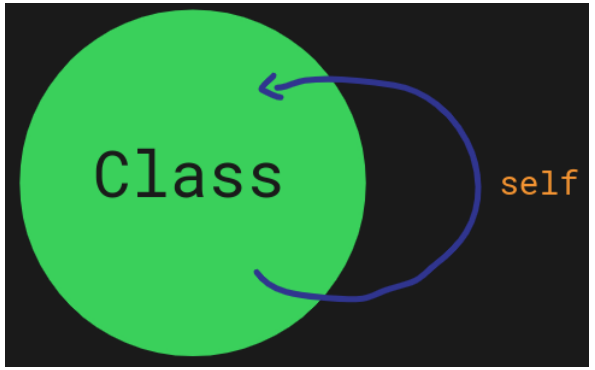
Object creation

Method call

```
cust = Customer()
cust.identify("Laura")
```

```
I am Customer Laura
```

- method definition = function definition within class
- **use `self` as the 1st argument in method definition**
- ignore `self` when calling method on an object



Using Methods

- A method call triggers a method's execution
  - As with functions, the flow of execution jumps to the method, performs the statements it contains, and then returns to the previous execution path

- Syntax to call a method that returns a value:

```
return_value = object.method(argument_1, argument_2, ...)
```

- Syntax to call a method that does not return a value:

```
object.method(argument_1, argument_2, ...)
```

Default Parameters

- Methods, like functions, can have default parameters

```
def method_name(self, param_1, param_2, optional_1=5, optional_2="hi"):
```

  - Parameters without default values must appear first
  - Parameters without default values require arguments when the method is called

```
def test(a, b=10):  # ✅ valid
def test(a=10, b):  # ❌ invalid
```

Using Constructors

- The __init__() initialization method is a special method that runs automatically when an object is created.
- Used to initialise object attributes

Constructor with parameters

```
class Person:
    def __init__(self, name):
        self.name = name

p1 = Person("Mike")
print("Hello, my name is "+ p1.name)
```

Output

```
Hello, my name is Mike
```

Constructor with default parameters

```
class Person:
    def __init__(self, name="Guest"):
        self.name = name

p1 = Person()
print("Hello, my name is "+ p1.name)
```

Output

```
Hello, my name is Guest
```

Magic Methods (Dunder Methods)

- These are special methods with **double underscores** before and after their name
  - E.g. __init__, __str__, __add__
- They let us customize how objects behave
- Python calls them automatically in certain situations
  - E.g., the __str__ method is called when we use the built in print()

```python
class Person:
    def __init__(self, name="Guest"):
        self.name = name

p1 = Person()
print(p1)
```

Output

```
<__main__.Person object at 0x797b1f1adc40>
```

```python
class Person:
    def __init__(self, name="Guest"):
        self.name = name

    def __str__(self):
        return "Person name is "+self.name

p1 = Person()
print(p1)
```

Output

```
Person name is Guest
```

Common Dunder Method

Dunder method	Description	Example
__init__(self, ….)	Constructor (runs when object is created)	P=Person(“Alice”)
__str__(self)	Defines string representation of object	print(p)
__len__(self)	Defines behaviour for len(object)	len(p)
__add__(self, other)	Defines behaviour for + operator	obj1 + obj2
__eq__(self, other)	Defines behaviour for == comparison	obj1 == obj2

Example

```python
class Person:
    def __init__(self, name, age):
        """Constructor: Initializes name and age"""
        self.name = name
        self.age = age

    def __str__(self):
        """String representation of the object"""
        return f"Person(name={self.name}, age={self.age})"

    def __len__(self):
        """Return the length of the person's name"""
        return len(self.name)

    def __add__(self, other):
        """Combine ages when adding two Person objects"""
        if isinstance(other, Person):
            return self.age + other.age
        return NotImplemented

    def __eq__(self, other):
        """Compare two Person objects based on name and age"""
        if isinstance(other, Person):
            return self.name == other.name and self.age == other.age
        return False


# Example usage:
person1 = Person("Alice", 25)
person2 = Person("Bob", 30)
person3 = Person("Alice", 25)

# __str__
print("Str: ",person1)  # Person(name=Alice, age=25)

# __len__
print("Length: ",len(person1))  # 5 (length of "Alice")

# __add__
print("Add: ",person1 + person2)  # 55 (25 + 30)

# __eq__
print("Comparison: ",person1 == person2)  # False
print("Comparison: ",person1 == person3)  # True
```

Output

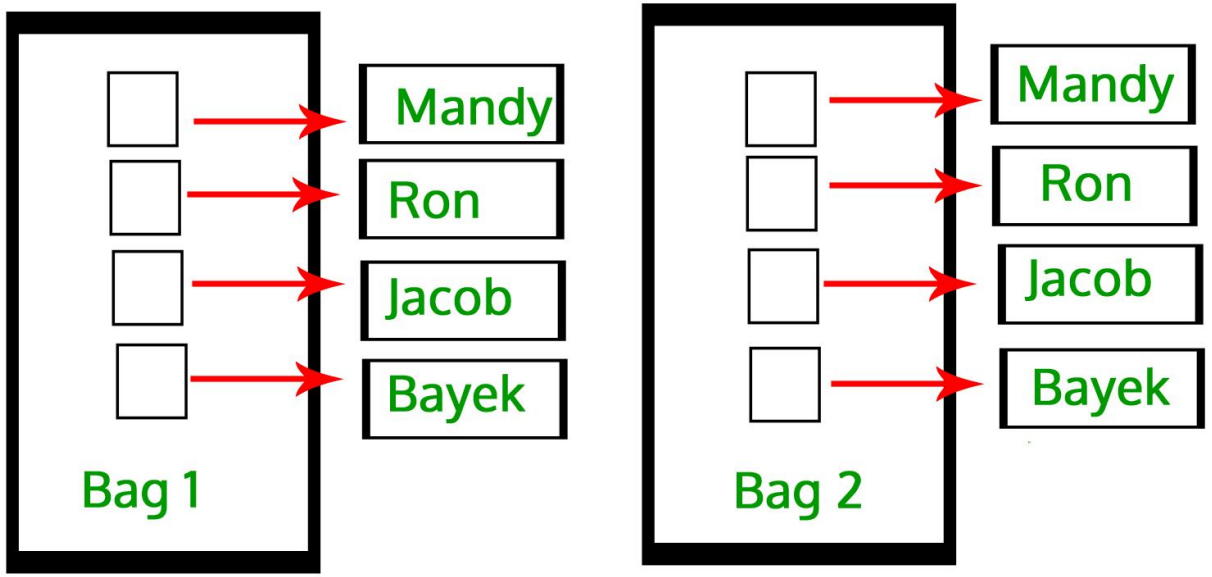
```
Str:  Person(name=Alice, age=25)
Length:  5
Add:  55
Comparison:  False
Comparison:  True
```

Object Assignment

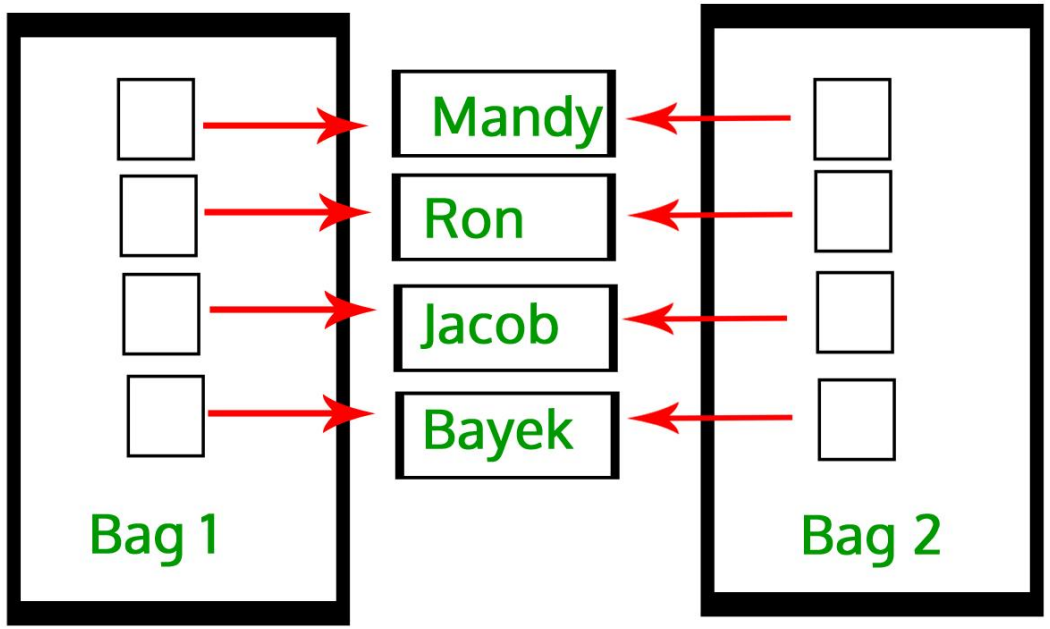
- Assignment operator does not create a copy of the object of the target
  - Instead creates a binding or link between target and the object
- = operator creates a reference of the object to the newly created variable
- copy module used to make a clone or real copy of an object.
- Two methods:
  - copy(): this method returns a shallow copy of the list
    - Syntax: copy.copy()
  - deepcopy(): this method returns a deep copy of the list
    - Syntax: copy.deepcopy()

Example

Deep Copy



Shallow Copy



Example

```python
original = [[1, 2, 3], [4, 5, 6]]

import copy
shallow = copy.copy(original)

shallow[0].append(99)

print("Original List: ",original)
print("Shallow Copy List: ",shallow)

print("\n")

deep = copy.deepcopy(original)

deep[0].append(77)
print("Original List: ",original)
print("Deep Copy List: ",deep)
```

Output Clear

```
Original List:  [[1, 2, 3, 99], [4, 5, 6]]
Shallow Copy List:  [[1, 2, 3, 99], [4, 5, 6]]


Original List:  [[1, 2, 3, 99], [4, 5, 6]]
Deep Copy List:  [[1, 2, 3, 99, 77], [4, 5, 6]]
```

Getters and Setters

- Getter:
  - Method to access (read) a private attribute
  - Also called as Accessor methods
- Setter:
  - Method to modify (update) a private attribute
  - Also called as Mutator methods

- Help enforce encapsulation
- Provide controlled access to data

Example

```python
class Person:
    def __init__(self, name, age):
        self.__name = name    # private attribute
        self.__age = age      # private attribute

    # Getter for age
    def get_age(self):
        return self.__age

    # Setter for age
    def set_age(self, new_age):
        if new_age > 0:
            self.__age = new_age
        else:
            print("Invalid age")

person1 = Person("Alice", 25)
print("Original age: ",person1.get_age())  # 25
person1.set_age(30)
print("Modified age: ",person1.get_age())  # 30
```

Output

```
Original age:  25
Modified age:  30
```

Why Use Them?

- Protects data from invalid updates
- Hides internal implementation from user (encapsulation)
- Provides flexibility for adding validation logic later
- Keeps code clean and secure

Next Week – What's Coming Up?

- Explain the role of inheritance in building a class hierarchy
- Explore the types of inheritance and necessary components
- Explore how polymorphism is used in OOP
- Define pure virtual method
- Dealing with Exceptions
- Using Exceptions

CIHE

CIHE Australia